

Pytest Fixtures

1 – Introdução a fixtures do pytest

Objetivo

Compreender o conceito de fixture no pytest e como utilizá-las para configurar e limpar ambientes de teste automaticamente.

Explicação

Fixtures são funções especiais do pytest usadas para preparar recursos antes do teste e liberar esses recursos depois.

Elas substituem setups manuais repetidos e ajudam a manter os testes limpos e organizados.

No Playwright, usamos fixtures para criar o browser, o contexto e a página, garantindo que cada teste receba o ambiente certo e que tudo seja fechado corretamente no final.

O uso de fixtures melhora a legibilidade e a manutenção dos testes, pois centraliza a configuração comum em um só lugar.

2 – Criando as fixtures browser, context e page

Objetivo

Criar fixtures para inicializar o browser, o contexto e a page do Playwright, gerenciando automaticamente o setup e o teardown.

Explicação

No Playwright, o fluxo natural para um teste é:

1. Criar o navegador (browser)
2. Criar um contexto (browser.new_context())
3. Criar uma página (context.new_page())

Usamos fixtures separadas para cada um desses níveis para facilitar o reuso e o controle do escopo.

No pytest com o plugin `pytest-playwright`, o próprio plugin já fornece a fixture `browser`, que abre o navegador para nós e aceita parâmetros via linha de comando, como `--browser chromium`, `--headed`, etc.

Isso é uma grande vantagem porque não precisamos criar essa fixture manualmente nem configurar o lançamento do browser — o pytest já faz isso, e nossa fixture só precisa usar o objeto que ele já criou.

Normalmente, o browser tem escopo `session` (é aberto uma vez por execução), já o contexto e a página têm escopo `function` (um por teste) para garantir isolamento.

Exemplo de código (conftest.py):

```
import pytest

@pytest.fixture(scope="function")
def context(browser):
    """Cria um novo contexto de navegador para cada teste, garantindo isolamento."""
    contexto = browser.new_context(
        base_url="https://automationexercise.com/",
        record_video_dir="videos" # Salva vídeos dos testes
    )
    yield contexto
    contexto.close()

@pytest.fixture(scope="function")
def page(context):
    """Cria uma nova página por teste, com tempo de espera configurado."""
    pagina = context.new_page()
    pagina.set_default_timeout(10000) # timeout padrão para ações
    pagina.set_default_navigation_timeout(30000) # timeout para navegação
    yield pagina
    pagina.close() # Opcional, pois o context fecha as pages também
```

Explicação do código

- `context` : cria um novo contexto de navegação a partir do browser. Isso garante que os cookies, cache e sessões não sejam compartilhados entre os testes.
- `page` : cria uma nova página dentro do contexto e define timeouts padrões para todas as interações.
- Ambas as fixtures usam `yield` para que o pytest saiba onde executar o teardown (fechamento dos recursos).

3 – Escopos de Fixtures

Objetivo

Entender o que são **escopos de fixtures** no `pytest` e como escolher o escopo ideal para cada tipo de recurso nos testes automatizados com **Playwright**, garantindo equilíbrio entre desempenho e isolamento.

Conceito

No `pytest`, o **escopo** define **por quanto tempo** a mesma instância de uma fixture será reutilizada durante a execução dos testes.

Existem quatro escopos possíveis:

`function`, `class`, `module`, `session`.

No curso vamos focar nos dois mais usados:

- `function` – Criada novamente **a cada teste**.
 - **Prós:** isolamento total entre testes.
 - **Contras:** mais lento quando o setup é pesado.
 - `session` – Criada **uma única vez** para todos os testes da execução.
 - **Prós:** mais rápido.
 - **Contras:** pode causar interferência entre testes se não houver isolamento.
-

Aplicação prática no Playwright

Escopos mais comuns

- **browser** → **session**
 - Aberto **uma vez só** e reutilizado em todos os testes.
 - Fixture já fornecida pelo **pytest-playwright**, sem necessidade de criar.
 - Aceita parâmetros via CLI:

```
pytest --browser chromium --headed
```

- **context** e **page** → **function**
 - Criam novo **contexto** e nova **aba** para cada teste.
 - Garantem isolamento de cookies, localStorage e sessão.

Exemplo de implementação no curso

```
# conftest.py
import pytest

@pytest.fixture(scope='function')
def contexto(browser):
    contexto = browser.new_context(
        base_url='https://automationexercise.com/',
        record_video_dir='videos'
    )
    yield contexto
    contexto.close()

@pytest.fixture(scope='function')
def page(contexto):
    pagina = contexto.new_page()
    pagina.set_default_timeout(10000)
    pagina.set_default_navigation_timeout(30000)
    yield pagina
    pagina.close()
```

Explicando os escopos nesse código

- **browser**: escopo **session** → aberto apenas uma vez.

- **contexto** e **page** : escopo **function** → criados e fechados a cada teste, evitando compartilhamento de estado.

Demonstração prática

Cenário 1 – Escopo **function** (isolamento garantido)

Dois testes rodam separados, sem influência entre si.

```
# test_login.py
from pages.cadastro_login import CadastroLogin
from playwright.sync_api import expect

def teste_login_valido(page):
    login = CadastroLogin(page)
    login.acessar_cadastro_login()
    login.fazer_login(email='teste@testeabcde.com', senha='123456789')
    expect(page.get_by_role("link", name="Logout")).to_be_visible()

def test_nao_logado(page):
    login = CadastroLogin(page)
    login.acessar_home()
    expect(page.get_by_role("link", name="Logout")).not_to_be_visible()
```

Resultado:

- No primeiro teste o usuário está logado.
- No segundo teste, a sessão começa limpa e o login não existe.

Cenário 2 – Escopo **session** (problema de estado compartilhado)

Se mudarmos:

```
@pytest.fixture(scope="session")
```

para **contexto** e **page**, os dois testes **compartilharão a mesma sessão**.

Problema:

- O segundo teste pode herdar o login do primeiro, quebrando o teste

Conclusão

- Use `session` para recursos que podem ser compartilhados com segurança (ex.: `browser`).
- Use `function` para recursos que exigem isolamento (ex.: `context` e `page`).
- Escolher o escopo correto impacta diretamente na **confiabilidade** e **velocidade** dos testes.

Referência oficial:

 <https://docs.pytest.org/en/6.2.x/fixture.html#fixture-scopes>